

SEQUENCING JOBS WITH UNEQUAL READY TIMES TO MINIMIZE MEAN FLOW TIME*

M. I. DESSOUKY† AND J. S. DEOGUN‡

Abstract. This paper presents a procedure for sequencing a set of jobs on a single processor (machine) with the objective of minimizing the mean flow time, when the jobs may have unequal ready times. The procedure involves implicit enumeration with the branch and bound technique, coupled with some devices to improve the efficiency of the search. The devices include a sufficient optimality condition, a simple but tight lower bound and rules for tree pruning. The approach has potential applications to other sequencing problems.

Key words. sequencing, scheduling, branch and bound

1. Introduction. The problem of sequencing n jobs on one processor or machine has been studied extensively under different assumptions and objective functions. In the simple problem of sequencing n jobs with equal ready times and no imposed due dates with the objective of minimizing the total flow time, it has been shown [2] that the shortest processing time (SPT) rule provides an optimal solution. According to this rule, jobs are sequenced from beginning to end on the basis of an ascending order of their processing times. One of the most constraining assumptions in this simple problem is the equality of the ready times of the jobs. The ready time of a job on a machine is the earliest time that the job can be made available for the machine to start processing it. This time is dictated by the completion time of the last operation conducted on the job at other machines, the arrival time of materials to the machine, the dispatching time of job instructions, and other relevant factors. It is conceivable that in the general case, the job ready times may not be identical.

The inequality of the ready times has been recognized in research on other single machine problems, where due dates were also taken into consideration [1], [3]. The objective of this paper is to develop an optimal scheduling procedure for the case when the ready times are not equal, and the total flow time is to be minimized. No due dates will be considered. Lenstra et al. [4] have shown that the problem is NP-complete; accordingly, an enumerative procedure for solving the problem will be needed.

2. Problem statement. A set N of n jobs, $N = \{i | i = 1, 2, \dots, n\}$, is to be processed, one job at a time, on a single processor (machine). For each job i , the ready time, r_i and the processing time, p_i , are given. Completion of all jobs requires establishing a sequence $S = (s_1, s_2, \dots, s_n)$, where s_y is the index number of the job in position y . When a job parameter or variable is identified by the job's position in a given sequence rather than its index number, the position is indicated as an underlined subscript to the parameter or variable. Thus r_4 means the ready time of the job in position 4 in the sequence considered.

Suppose that a sequence is constructed by adding one job at a time, starting from position 1. At any point, we have a partial sequence S_K of a job set $K \subseteq N$, $S_K = (s_1, \dots, s_k)$. The earliest start time of a job $i \in N$, R_i , and its completion time, C_i , are

* Received by the editors January 11, 1979, and in revised form January 23, 1980.

† Department of Mechanical and Industrial Engineering, University of Illinois, Urbana, Illinois 61807.

‡ Department of Computer Science, University of South Carolina, Columbia, South Carolina 29208.

given by

$$(1a-c) \quad R_i = \begin{cases} r_i & \text{if } i = s_1, \\ \max(r_i, C_{s_{y-1}}) & \text{if } i = s_y \in K, i \neq 1, \\ \max(r_i, C_k) & \text{if } i \in \bar{K} = N - K; \end{cases}$$

i 为 K 中非首位
 i 为 K 中末位

$$(2) \quad C_i = R_i + p_i.$$

For any job i , the flow time F_i and the waiting time W_i are defined as follows: $F_i = C_i - r_i$ and $W_i = R_i - r_i$. For a sequence S , the total completion time $C(S) = \sum_{i=1}^n C_i(S)$, the total flow time $F(S) = \sum_{i=1}^n F_i(S)$, and the total waiting time $W(S) = \sum_{i=1}^n W_i(S)$. Conway, Maxwell and Miller [2] show that a sequence S^0 which minimizes $C(S)$ will also minimize $F(S)$ and $W(S)$. In addition, the mean values $\bar{C}$, $\bar{F}$, and $\bar{W}$ are minimized. The purpose of this paper is to present a procedure for determining S^0 such that $C(S^0) = \min_S C(S)$. This problem, with equal or unequal ready times, is commonly called the $n/1/\bar{F}$ problem [2].

$\rightarrow n$ 个作业。单机，优化流转时间

3. Two sequencing rules. First, we present two rules for sequencing the n jobs which are utilized in the derivation of a procedure for solving the $n/1/\bar{F}$ problem addressed. Each rule specifies priority criteria for adding a job to an existing partial sequence, S_K , $K \subseteq N$, starting with position 1.

A. **The earliest completion time (ECT) rule:** Select job i with $\min_{i \in \bar{K}} C_i$. Break ties by choosing i with $\min R_i$, and further ties by choosing i with $\min i$. Update R_i and C_i , $i \in \bar{K}$, after each addition using (1c) and (3).

B. **The earliest start time (EST) rule:** Select job i with $\min_{i \in \bar{K}} R_i$. Break ties by choosing i with $\min C_i$, and further ties by choosing i with $\min i$. Update R_i and C_i , $i \in \bar{K}$ after each addition using (1c) and (2).

It can be easily shown that for any job set $K \subseteq N$, an EST sequence guarantees the minimum completion time for the last job in the set.

4. The approach. The proposed procedure involves a branch and bound search conducted along the branches of a tree in which a node at level k represents a partial sequence S_K of a set K of k jobs. For each node S_K , we compute a lower bound $\underline{C}(S^* | S_K)$ and an upper bound $\bar{C}(S^* | S_K)$ on $C(S^* | S_K)$, the minimal total completion time of any sequence starting with S_K , that is, conditional on S_K .

A node at level $k+1$ is formed by selecting a job $i \in \bar{K}$ and adding it to S_K in position $k+1$ to form (S_K, i) . At any iteration, the node being expanded is called the current node and has the current lowest lower bound. A closed (fathomed) node is one whose corresponding partial sequence has been found dominated, and, hence, is eliminated from consideration. Dominance is tested between nodes generated from the same parent. A partial sequence (S_K, j) is dominated if another partial sequence (S_K, i) exists and $\underline{C}(S^* | (S_K, i)) \leq \underline{C}(S^* | (S_K, j))$; it is strictly dominated if $\underline{C}(S^* | (S_K, i)) < \underline{C}(S^* | (S_K, j))$. We apply a number of tests (pruning rules or elimination criteria) to identify dominated nodes. A set V contains all active nodes ordered by nondecreasing lower bounds with the current node placed at its beginning. An active node is one which has not been found dominated.

5. Optimality and dominance properties. In this section, we present relevant properties of the $n/1/\bar{F}$ problem which are useful in testing a sequence for optimality, in defining lower bounds and in devising elimination criteria. We state the properties as theorems with proof given in the Appendix. We will identify R_i and C_i by the sequence in question whenever it is not clear from the context.

THEOREM 1. Given a job set N and a partial sequence S_K , $K \subset N$, if a job $i \in \bar{K} = N - K$ has $p_i \leq p_j$ all $j \in \bar{K}$, and a job $h \in \bar{K}$ has $R_h(S_K) \geq R_i(S_K)$, then i dominates h in position $k+1$. If $R_h(S_K) > R_i(S_K)$, then (S_K, i) strictly dominates (S_K, h) .

THEOREM 2. Consider a job set N of n jobs, and a partial sequence S_K of k jobs, $K \subset N$, and let job $i \in \bar{K}$ have $C_i(S_K) \leq C_j(S_K)$ all $j \in \bar{K}$. A partial sequence (S_K, j) is strictly dominated if $r_j \geq C_i(S_K)$.

THEOREM 3. Given S_K , $K \subseteq N$, and two jobs $i, j \in \bar{K}$, if $p_j \leq p_i$ and $C_j(S_K) \geq C_i(S_K)$, then (S_K, i) dominates (S_K, j) .

In a sequence S , define a block $b \subseteq S$ as a set of consecutive jobs with the first job s_u having $r_u > C_{u-1}(S)$ and all other jobs $s_w \in b$ having $r_w \leq C_{w-1}(S)$.

THEOREM 4. Let S^c be an ECT sequence of the set N consisting of one block with job i at the beginning and S^a be another sequence of the same set starting with job j . If $r_i = r_j$, then $C(S^c) \leq C(S^a)$; and if $r_i < r_j$, then $C(S^c) < C(S^a)$.

COROLLARY 4. An ECT sequence of a job set is optimal if it consists of a single block and the first job has the smallest ready time.

THEOREM 5. A sufficient condition for the optimality of a sequence is that every block is ordered according to ECT and starts with the job having the smallest ready time.

6. The algorithm. The algorithm consists of three basic phases: initialization, branching and termination. Initialization involves defining the initial values of the variables, constructing an initial sequence and testing the sequence for optimality.

② Branching is an iterative procedure; in each iteration it generates the descendants of the current node, eliminates dominated nodes, computes lower and upper bounds for active nodes, updates the file of active nodes and identifies the next current node, the one with the least lower bound. The upper bound at a node is the value of a complete sequence starting with the node's partial sequence. A noncurrent node is dominated if its lower bound is not smaller than the current lowest upper bound, which is the best solution reached thus far. The termination phase consists of identification of the optimal sequence S^0 and computation of $C(S^0)$ and $F(S^0)$.

Since the procedure for determining the lower and upper bounds at a node S_K is used repetitively in the algorithm, we list it separately below as procedure BOUND. To compute the upper bound $\bar{C}(S^* | S_K)$, we construct a complete sequence S . We define S as $S = (S_K, S_{\bar{K}})$, where $S_{\bar{K}}$ is an ECT sequence of $\bar{K}$ following S_K . We later demonstrate that S is generally a tight upper bound. To determine the lower bound $\underline{C}(S^* | S_K)$, we construct a sequence $S' = (S_K, S_{\bar{K}}')$ which is similar to S above, in the order of jobs and their processing times, but with the ready times partially relaxed. To construct S' given S_K , schedule each job s'_{k+h} , $1 \leq h \leq n - k$ (s'_{k+h} is identical to s_{k+h} in S) to start at the completion time of the preceding job s'_{k+h-1} , or at the minimum ready time of unscheduled jobs, including s'_{k+h} itself, whichever is greater. The sum of completion times of the jobs in S' , $C(S')$, is a lower bound at S_K , $\underline{C}(S^* | S_K)$. This statement is proven in Theorem 6.

I. Initialization.

1. Let N be the set of n jobs $\{i | i = 1, 2, \dots, n\}$. Define the parameters r_i and p_i for all $i \in N$.

2. Set $k = 0$, $V = K = \phi$, $S_K = \phi$, $\bar{K} = N - K$ and $C_k(S_K) = C(S_K) = 0$.

3. To construct an ECT sequence S^c of N and compute upper and lower bounds, define $y = k$, $Y = K$, $\bar{Y} = \bar{K}$, $S_Y = S_K$, $C(S_Y) = C(S_K)$ and $C'_y(S_Y) = C_y(S_Y) = 0$. Go to procedure BOUND. The lower bound $\underline{C}(S^0) = \text{LB}$, the upper bound $\bar{C}(S^0) = \text{UB}$ and $S^c = S_y$. Set the lowest upper bound $\text{LUB} = \text{UB}$.

4. Apply the optimality test of Theorem 5 to S^c . If S^c passes the test, go to step 12; otherwise, proceed to step 5.

由 E 结点生成的新序列

可调度作业集合

II. Branching.

符合条件的

5. Define the set of jobs eligible for adding to S_K as $E = \bar{K}$. Compute $R_j(S_K) = \max(r_j, C_k(C_K))$, $C_j(S_K) = R_j(S_K) + P_j$, and $C_m(S_K) = \min_{j \in \bar{K}} C_j(S_K)$. Eliminate all j from E for which $r_j \geq C_m(S_K)$ (Theorem 2).

6. Identify v satisfying $p_v = \min_{j \in \bar{K}} p_j$. If $v \in E$, eliminate all $j \in E$ for which $R_j(S_K) \geq R_v(S_K)$ (Theorem 1).

7. Eliminate j from E if some $i \in E$ exists for which $p_i \geq p_j$ and $C_i(S_K) \leq C_j(S_K)$ (Theorem 3).

8. For each $i \in E$, determine the lower and upper bounds at (S_K, i) and the conditional ECT sequence S^i as follows: set $y = k + 1$, $Y = \{K \cup i\}$, $\bar{Y} = \{\bar{K} - i\}$, $S_Y = (S_K, i)$, $C(S_Y) = C(S_K) + C_i(S_K)$ and $C'_y(S_Y) = C_y(S_Y) = C_i(S_K)$. Go to procedure BOUND. We obtain $\underline{C}(S^*|(S_K, i)) = \text{LB}$, $\bar{C}(S^*|(S_K, i)) = \text{UB}$, and $S^i = S_Y$. If $\text{LB} \geq \text{UB}$, eliminate i from E ; otherwise, set $\text{LUB} = \min(\text{LUB}, \text{UB})$.

9. Remove node S_K from V , and add nodes (S_K, i) , all $i \in E$. Store $C_i(S_K)$, $\underline{C}(S^*|(S_K, i))$, $\bar{C}(S^*|(S_K, i))$.

10. Identify the node S_Q in V with the least lower bound. Let the number of jobs in the set Q be q . Set $k = q$, $K = Q$, $\bar{K} = N - K$, $S_K = S_Q$, $C(S_K) = C(S_Q)$ and $C_k(S_K) = C_q(S_Q)$. Let the last job in S_K be i .

11. To recreate the conditional ECT sequence S^i , set $y = k$, $Y = K$, $\bar{Y} = \bar{K}$, $S_Y = S_K$, $C(S_Y) = C(S_K)$ and $C_y(S_Y) = C_i(S_Y)$. Go to procedure BOUND. If $\bar{K} =$ or $\underline{C}(S^*|S_K) = \text{LB} = \text{UB} = \bar{C}(S^*|S_K)$ proceed to step 12; otherwise, go to step 5.

III. Termination.

12. Let $S^0 = S^i$, $C(S^0) = \bar{C}(S^*|S_K)$, and $F(S^0) = C(S^0) - \sum_{i=1}^n r_i$.

Terminate.

Now to determine LB, UB and S^i at a node representing a partial sequence S_Y of y jobs ending with job i , follow procedure BOUND outlined below. The variables $C(S_Y)$, $C'_y(S_Y)$ and $C_y(S_Y)$ have already been defined in the calling step of the main algorithm.

Procedure BOUND. 当前处理的 $\rightarrow$ 进入定界的初始任务集

B1. Initialize: $x = y$, $X = Y$, $S_X = S_Y$, $\text{LB} = \text{UB} = C(S_Y)$ and $C'_x = C_x = C_y(S_Y)$.

B2. Let $m_{x+1} = \min_{j \in \bar{X}} r_j$ and compute $R_j = \max(C_x, r_j)$, and $C_j = R_j + p_j$, all $j \in \bar{X}$.

B3. Select $h \in \bar{X}$ such that $C_h = \min_{j \in \bar{X}} C_j$. Break ties by choosing h to minimize R_h .

B4. Place h in position $x + 1$ and set $R'_{x+1} = R'_h = \max(C_x, m_{x+1})$, $C'_{x+1} = C'_h = R'_{x+1} + P_h$, $P_{x+1} = R_h$, $C_{x+1} = C_h$, $\text{LB} = \text{LB} + C'_{x+1}$, and $\text{UB} = \text{UB} + C_{x+1}$.

B5. Set $x = x + 1$, $X = \{X \cup h\}$, $\bar{X} = \{\bar{X} - h\}$, and $S_X = (S_x, h)$.

B6. If $x = n$, set $S = S_X$; $\underline{C}(S^*|S_Y) = \text{LB}$; and $\bar{C}(S^*|S_Y) = \text{UB}$ and return to calling step; otherwise, go to step B2.

The sequence defining the upper bound UB is $S = (S_Y, S_{\bar{Y}})$, an ECT ordering of $\bar{Y}$. Now let $S' = (S_Y, S_{\bar{Y}'})$ be the sequence defining the lower-bound LB; that is, $\text{LB} = C(S')$. It may be seen that S and S' are identical in their job order but that their job start and completion times may be different. Theorem 6 establishes that $C(S')$ is a lower bound on the optimal solution conditional on S_Y .

THEOREM 6. $C(S') = C(S_Y, S_{\bar{Y}'}) \leq C(S^*|S_Y)$.

7. Example. This example is especially constructed to demonstrate most of the features of the procedure. Table 1 exhibits the job set parameters. For simplicity, we have ordered the jobs by ECT. Detailed computations for the first iteration are given below, where the step numbers correspond to those in the algorithm.

1. $n = 5$, and values of r_i and p_i for $i = 1, 2, \dots, 5$ are given in Table 1.

2. Set $k = 0$, $K = \phi$, $S_K = \phi$ and $\bar{K} = (1, 2, 3, 4, 5)$. Set $C_k = C(S_K) = 0$.

找出可
调度节点
集合E

剪枝

定界

关闭节点

找出下一个

扩结点

对已有

序列构建

ECT重新

计算上下界

判断

下个扩展结点

为当前确定完上下界明序列 S_k 构造 ECT

7

下一个分支点

理想状态

TABLE 1. Job set parameters

Job Indices	Ready Times	Processing Times
1	35	3
2	22	18
3	34	17
4	37	21
5	66	25

3. $S^c = (1, 2, 3, 4, 5)$. From procedure BOUND, $\underline{C}(S^*|S_K) = \text{LB} = 205$ and $\bar{C}(S^*|S_K) = \text{UB} = 386$.

4. The optimality test of Theorem 6 fails because the first job does not have the minimum ready time.

5. $E = (1, 2, 3, 4, 5)$. $C_1 = 38$ is the minimum among all $C_j, j \in E$. $R_5 = 66 > C_1 = 38$. Therefore, removing job 5 from E (Theorem 2), $E = (1, 2, 3, 4)$.

6. Job 1 has the shortest p_i , $p_1 = 4$. Since $r_4 = 37 > r_1 = 3$, eliminate 4 from E (Theorem 1). $E = (1, 2, 3)$.

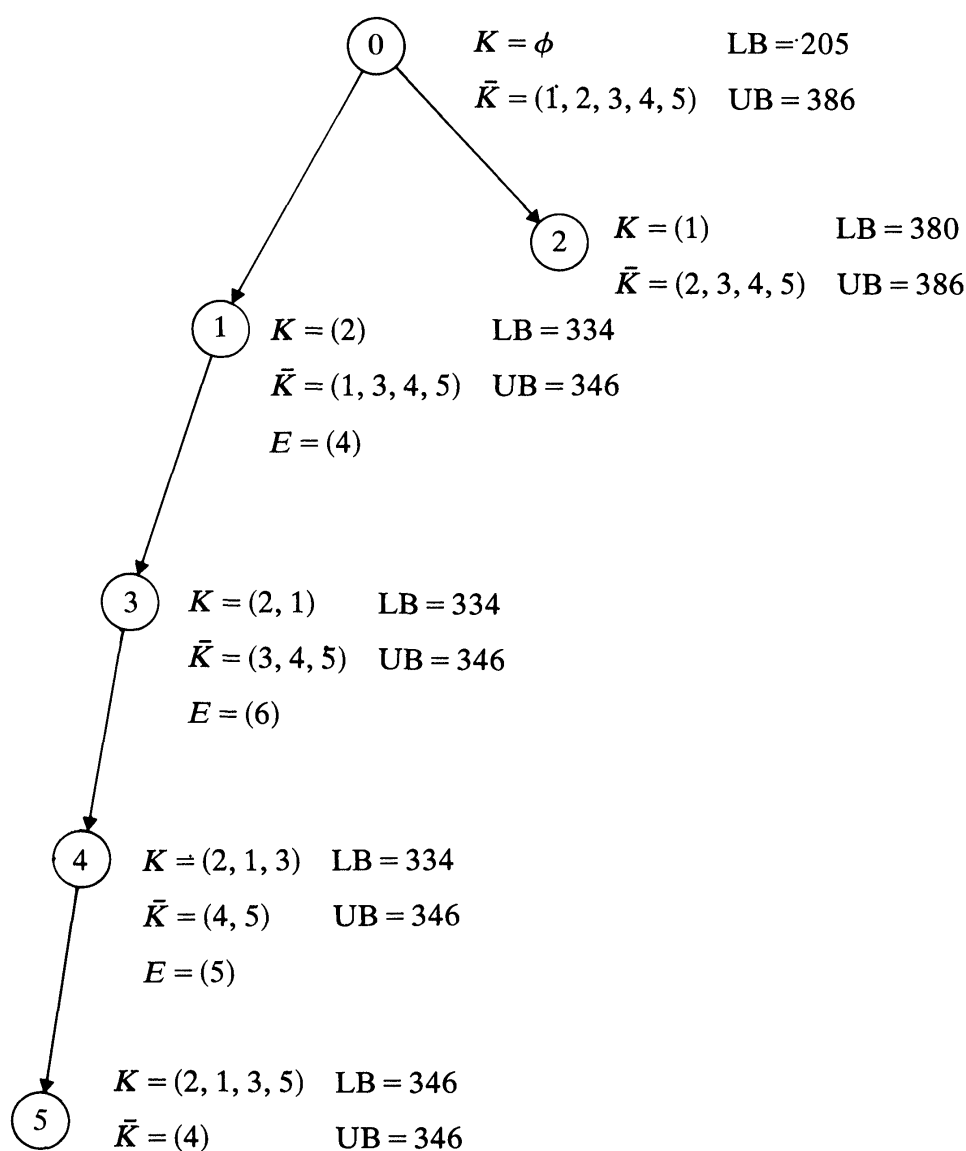


FIG. 1

7. $C_3 = 54$, $>C_2 = 40$ and $p_3 = 17 < p_2 = 18$, hence job 2 dominates job 3 (Theorem 3). Remove job 3 from E , $E = (1, 2)$.

8. Compute lower and upper bounds by procedure BOUND. $LB(1) = \underline{C}(S^*|(1)) = 380$ and $UB(1) = \bar{C}(S^*|(1)) = 386$. Similarly, $LB(2) = \underline{C}(S^*|(2)) = 334$ and $UB(2) = \bar{C}(S^*|(2)) = 346$. The lowest upper bound LUB is updated as follows: $LUB = \min(386, 346) = 346$. As $\underline{C}(S^*|(S_K, 1)) = 380$ $LUB = 346$. Remove job 1 from E . $E = (2)$.

9. Node (2) is added to set V .

10. Locate the node in V with the lowest lower bound. Set $y = 1$, $Y = K = \{2\}$, $\bar{Y} = \bar{K} = \{1, 3, 4, 5\}$.

11. Using procedure BOUND, reconstruct $S_{\bar{K}} = (1, 3, 4, 5)$ and $LB(2) = \underline{C}(S^*|(2)) = 334$ and $UB(2) = \bar{C}(S^*|(2)) = 346$. Neither $\bar{K} = \phi$, nor $LB(2) = UB(2)$ therefore, proceed to step 5.

This completes the first iteration in the branch and bound algorithm. The complete tree of optimal solutions generated is shown in Fig. 1. The optimal sequence is (2, 1, 3, 5, 4) and the minimum value of the total completion time is 346. The minimum total flow time is 152.

8. Experimental evaluation. The algorithm was programmed and tested on a Cyber 175, using a FORTRAN G compiler. Two tests were conducted, the first test consisting of 220 job sets, each generated randomly from two independent probability distributions of r_i and p_i . Four distributions for r_i and eleven for p_i combined to produce 44 job set distributions, each generating five problems (job sets of 20 jobs each). The second test consisted of 44 job sets, 50 jobs each, with each job set from each of the forty-four distributions above. The probability distributions were chosen to be uniform to avoid biasing particular values within their ranges, and to eliminate the need for truncation. They are specified by the upper and lower limits of their ranges. We selected the distributions to include the ranges which are most challenging to the algorithm.

Table 2 shows the experimental results for all problems in the first test. Out of the 220 problems, 214, 215 and 217 were solved within $\frac{1}{2}$, 1 and 2 seconds respectively. Within the two-second limit, the mean and maximum computation times were .05 and 1.88 seconds, the mean and maximum number of nodes generated were 13 and 514, and the mean and maximum number of iterations were 9 and 497. The three problems which were not solved within two seconds were later run to completion. The maximum computation time for any problem was 4.61 seconds. The mean time to solve all 220 problems was 0.08 seconds. Comparing the optimal solution for each of the three problems with that obtained within two seconds, the mean and maximum per cent deviation from optimality were 17.6% and 31%. Averaged over all problems, the mean per cent deviation was 0.24%.

Apart from these summaries, Table 2 also indicates that the performance of the algorithm deteriorates as the ratio of the range of r_i to the mean of p_i exceeds 2.5.

In the second test all problems were run to completion. Table 3 shows the experimental results for the second test. Out of 44 problems, 30, 39 and 40 problems were solved within $\frac{1}{2}$, 1, and 2 seconds. The mean and maximum computation times were .96 and 8.47 seconds, the mean and maximum number of nodes generated were 47 and 640, and the mean and maximum number of iterations were 42 and 484. These results demonstrate that the algorithm can be effectively used to solve large problems.

The effectiveness of ECT sequencing may be demonstrated by the fact that in 173 out of the 220 problems the initial sequence was optimal, although it was recognized as optimal in only 53 problems. The mean deviation from optimality of the initial ECT

TABLE 2
Summary of results: Branch and bound—ECT procedure for $N/1/\bar{F}$ (20 jobs in each set)

Range of ready times	Range of processing times										
	1-25	26-50	1-75	51-75	1-125	76-100	1-175	101-125	1-225	126-150	1-275
0-200	***	2 1	8 8	1 0	25 19	1 1	2 2	0 0	1 2	1 1	0 0
	***	3 2	17 14	3 4	17 14	1 1	8 10	0 0	3 4	1 1	0 0
	64 24	2 1	13 9	0 1	7 9	0 0	11 13	1 1	3 4	0 0	3 4
	178 43	1 0	7 3	1 0	7 7	0 0	7 10	1 1	1 1	0 0	1 1
	3 1	1 0	9 8	0 1	1 1	1 0	2 4	0 0	1 2	0 1	1 3
	Min 1	Min 0	Min 3	Min 0	Min 1	Min 0	Min 2	Min 0	Min 1	Min 0	Min 0
	Max 43	Max 2	Max 14	Max 4	Max 19	Max 1	Max 13	Max 1	Max 4	Max 1	Max 4
25-175	Mean 13	Mean 0	Mean 8	Mean 1	Mean 10	Mean 0	Mean 7	Mean 0	Mean 2	Mean 0	Mean 1
	13 5	4 4	12 8	1 1	30 23	0 0	9 14	1 1	4 4	0 1	10 10
	71 43	9 6	25 17	1 1	5 3	1 0	7 6	0 0	8 12	0 0	4 6
	25 5	5 6	***	1 0	23 23	1 1	4 5	0 0	3 4	0 0	1 1
	395 150	0 0	2 1	1 1	1 1	0 0	1 2	1 2	3 4	0 0	1 4
	497 188	0 0	11 5	1 2	1 2	1 2	2 3	2 3	3 3	0 0	3 4
	Min 5	Min 0	Min 1	Min 0	Min 1	Min 0	Min 2	Min 0	Min 3	Min 0	Min 1
50-150	Max 188	Max 6	Max 17	Max 2	Max 23	Max 2	Max 14	Max 3	Max 12	Max 1	Max 10
	Mean 78	Mean 3	Mean 6	Mean 1	Mean 10	Mean 0	Mean 6	Mean 1	Mean 5	Mean 0	Mean 4
	102 50	1 1	7 9	0 0	3 4	1 1	1 2	0 0	3 3	0 0	3 5
	18 8	2 3	14 10	0 0	8 10	0 1	2 2	0 0	6 10	0 1	3 5
	6 3	1 1	10 9	0 1	2 2	1 1	1 2	0 0	1 1	0 0	2 3
	3 1	1 1	13 12	1 1	12 17	1 3	2 1	0 0	2 4	0 0	1 1
	9 7	1 1	6 6	0 0	1 1	1 1	0 1	0 0	1 1	0 1	1 1
75-175	Min 1	Min 1	Min 6	Min 0	Min 1	Min 1	Min 1	Min 0	Min 1	Min 0	Min 1
	Max 50	Max 3	Max 12	Max 1	Max 17	Max 3	Max 2	Max 0	Max 10	Max 1	Max 5
	Mean 13	Mean 1	Mean 9	Mean 0	Mean 6	Mean 1	Mean 1	Mean 0	Mean 3	Mean 0	Mean 3
	90 47	1 1	7 11	0 0	6 7	1 2	0 0	1 2	1 1	0 1	0 0
	58 21	0 0	2 1	1 0	27 36	0 0	2 2	1 2	1 2	0 0	2 2
	16 9	1 1	4 4	1 0	4 4	0 1	1 1	1 2	2 2	1 2	0 0
	3 2	1 1	19 21	1 2	0 0	0 0	1 2	1 3	1 2	0 0	2 1
	25 14	0 0	2 1	1 1	1 2	1 1	2 3	1 2	3 5	0 1	3 6
	Min 2	Min 0	Min 1	Min 0	Min 0	Min 0	Min 0	Min 2	Min 1	Min 0	Min 0
	Max 47	Max 1	Max 21	Max 2	Max 36	Max 2	Max 3	Max 3	Max 5	Max 2	Max 6
	Mean 18	Mean 0	Mean 7	Mean 0	Mean 9	Mean 0	Mean 1	Mean 2	Mean 2	Mean 0	Mean 1
	Min 1	Min 0	Min 1	Min 0	Min 0	Min 0	Min 0	Min 0	Min 1	Min 0	Min 0
	Max 188	Max 6	Max 21	Max 4	Max 36	Max 3	Max 14	Max 3	Max 12	Max 2	Max 10
	Mean 30	Mean 1	Mean 7	Mean 0	Mean 8	Mean 0	Mean 3	Mean 0	Mean 3	Mean 0	Mean 2

Number of jobs = 20

Minimum computation time = 0 Minimum number of iterations = 0
 Maximum computation time = 188 Maximum number of iterations = 497
 Mean computation time = 5 Average number of iterations = 9
 Number exceeding $\frac{1}{2}$ second = 6 Minimum number of nodes generated = 1
 Number exceeding one second = 5 Maximum number of nodes generated = 514
 Average number of nodes generated = 13

Note: For each box, the first column is the number of iterations and the second column is computer time in centi-seconds. The *'s in a column show that the corresponding problem was not solved in 2 seconds.

sequence was 3% for all problems. Thus, the use of ECT sequencing provides fairly tight upper bounds which can help to eliminate many dominated nodes.

9. Conclusions and recommendations. This paper presents an analysis of optimality and dominance conditions for the $n/1/\bar{F}$ problem with unequal ready times and a procedure for solving the problem. Although the proofs of the theorems are rather lengthy, the procedure itself is simple. Experimentation with the algorithm on a Cyber 175 computer showed in the first test that for 220 job sets with 20 jobs each, the

TABLE 3
Summary of results: Branch and bound—ECT procedure for $N/1/\bar{F}$ (50 jobs in each set)

Range of ready times	Range of processing times											
	1-25	26-50	1-75	51-75	1-125	76-100	1-175	101-125	1-125	126-150	1-275	
0-200	308 633	3 72	22 162	1 2	38 81	1 41	5 55	1 3	5 11	1 42	11 67	Min 2 Max 633 Mean 106
25-175	455 773	10 12	28 76	1 3	37 92	2 4	5 48	0 2	31 87	0 39	11 18	Min 2 Max 773 Mean 104
50-150	313 847	1 3	14 100	1 5	12 50	3 7	19 11	0 1	6 52	0 41	8 21	Min 1 Max 847 Mean 103
75-175	484 553	1 5	7 50	1 2	4 48	1 3	6 49	1 2	4 47	1 5	2 7	Min 2 Max 653 Mean 70
Min	553	3	50	2	48	3	11	1	11	5	7	
Max	847	72	162	5	92	41	55	3	87	42	67	
Mean	701	23	97	3	67	13	40	2	49	31	28	

Number of jobs = 50

Minimum computation time = 1
Maximum computation time = 847
Mean computation time = 96
Number exceeding $\frac{1}{2}$ second = 14
Number exceeding 1 second = 5

Minimum number of iterations = 0
Maximum number of iterations = 484
Average number of iterations = 42
Minimum number of nodes generated = 1
Maximum number of nodes generated = 540
Average number of nodes generated = 47

Note: For each range of processing time, the first column is the number of iterations and the second column is computer time in centi-seconds. The statistics Min, Max and Mean are shown for computer time only.

algorithm was capable of solving any problem within a maximum of 4.61 seconds of computer time and that all but 3 out of 220 problems were solved within two seconds, with an overall average of 0.08 seconds.

The second test demonstrated the effectiveness of the algorithm to solve large problems. For 44 job sets with 50 jobs each, in the second test, the algorithm solved any problem within a maximum of 8.47 seconds and an overall average of .96 seconds.

The algorithm can be used to test alternative heuristics for closeness to optimality. It can also be applied to two and multiple machine problems in the process of computing lower bounds on the optimal solution to those problems.

Minimizing the sum of completion times is only one of many interesting functions that could be considered. However, only this function has been addressed in this paper because it gives a simple measure of the effectiveness of a schedule and it yields an algorithm which may be extended to solve more complex functions. Natural extensions are to minimize the sum of weighted completion times, and to minimize the maximum flow time.

Appendix. Proofs of theorems.

Proof of Theorem 1. Any sequence $S^h = (S_K, S_K^h)$ having h in position $k + 1$ and i in position y , $y > k + 1$, will have $r_i \leq R_i(S_K) \leq R_h(S_K) = R_{k+1}(S_K) \leq R_x(S^h)$, $k < x < y$.

Interchanging i with its immediate predecessor in position $y - 1$ will not increase the completion time of the jobs in positions $y - 1$ and y , nor that of any preceding or succeeding job in S^h . Repeating the interchange backward to position $k + 1$ following S_K will produce a sequence S^i having $C_x(S^i) \leq C_x(S^h)$, $1 \leq x \leq n$, and $C(S^i) \leq C(S^h)$; that is, (S_K, i) dominates (S_K, h) . Similarly, we can show that if $R_h(S_K) > R_i(S_K)$, then (S_K, i) strictly dominates (S_K, h) . Q.E.D.

Proof of Theorem 2. Let S^j be the optimal sequence of N given that j is in position $k + 1$ following S_K . An idle time gap equal to $r_j - C_k(S^j)$ will exist on the machine between the jobs in positions k and $k + 1$. If job i is shifted from its position, say g , to position $k + 1$ without changing the order of any other job, it will occupy no more of the machine time than the gap and will gain a reduction in its completion time. Now form S^i starting with S_K , followed by i , then j , then the remaining jobs in the same order of S^j . Shifting the jobs in positions $g + 1$ to n as early as possible may reduce their completion times without influencing the completion times of the jobs in positions $k + 2$ to g , which remain the same. Therefore, $C(S^i) < C(S^j)$ and (S_K, j) is strictly dominated. Q.E.D.

Proof of Theorem 3. Let S^j be an optimal sequence of N given that j is in position $k + 1$ following S_K . Consider removing jobs i and j from the schedule. Without affecting the completion times of any other jobs, job i may be scheduled in position $k + 1$ as $C_i(S_K) \leq C_j(S_K)$; and job j may be scheduled in i 's former position as $p_j \leq p_i$. The total completion time of jobs i and j is not increased as neither is completed later in the new schedule S^i than the other was in S^j . Thus $C(S^i) \leq C(S^j)$. Q.E.D.

Proof of Theorem 4. Assume that the ready time of each job $y \in N$ is modified to $r'_y = \max(r_y, r_i)$. This assumption will not affect the start or completion time of any job in S^c or S^a , as no job in either sequence starts before r_i . 证明:

Consider the case when $r_i = r_j$. First, we show that $C_k(S^c) \leq C_k(S^a)$, $1 \leq k \leq n$. For a given k , reorder the first k jobs in S^a according to their earliest start times (EST), assuming modified ready times, and breaking ties by giving priority to the job with the smallest completion time among tied jobs. Call the resulting sequence S_K^t ; as mentioned in § 3, $C_k(S^t) \leq C_k(S^a)$. Note that S^c is both an ECT and an EST sequence of the modified set N . Therefore, at the first position x , $1 \leq x \leq k$ in which $C_x(S^c) \neq C_x(S^t)$, it must be true $C_x(S^c) < C_x(S^t)$ and $s_x^c \notin S_K^t$. Therefore, replacing s_k^t with s_k^c will not result in an increase in any $C_y(S^t)$, $1 \leq y \leq k$. This replacement may cause the set of jobs in S_K^t following the new s_k^c to lose its EST sequencing property. In that case, reordering the set by EST will not increase $C_k(S^t)$, since $C_x(S^t)$ has been reduced. Redefine S_K^t to describe the new sequence. Repeating this procedure for all y , $x < y \leq k$, comparing C_y^c and C_y^t and making necessary changes as shown above will produce a sequence identical to the partial sequence of the first k jobs in S^c , without increasing C_k^t . Thus $C_k^c \leq C_k^t \leq C_k^a$, $1 \leq k \leq n$ and $C(S^c) = \sum_{k=1}^n C_k^c \leq \sum_{k=1}^n C_k^a = C(S^a)$, proving the first part of the theorem.

For $r_i < r_j$, applying the same procedure yields $C_k^c \leq C_k^a$, $1 \leq k \leq n$. From Theorem 3, $C_i \leq C_j$ implies that if $p_i > p_j$ then $C(S^c) < C(S^a)$ and the theorem is proved. If $p_i \leq p_j$ then $C_1^c = C_1^c < C_j^a = C_1^a$, and $C(S^c) < C(S^a)$. Q.E.D.

Proof of Theorem 5. Let the number of blocks be B . If $B = 1$, the theorem is true by Corollary 4. Suppose that the theorem is true for the first b blocks where $1 \leq b < B$. From the definition of a block and the conditions of the theorem, $r_j > C_i(S)$ for all j in block $b + 1$, where i is the last job in the optimal sequence of the first b blocks. By Theorem 2, all jobs in block b are dominated by job i ; therefore, for optimality, no jobs in block $b + 1$ should be scheduled earlier and block $b + 1$ can be optimized separately and then combined to form a complete optimal sequence for $b + 1$ blocks. By induction, all B blocks can be optimized separately. Since each block satisfies the optimality conditions of Corollary 4, then the sequence is optimal. Q.E.D.

Proof of Theorem 6. First, we define the parameters of the set of jobs $\bar{Y}'$ in $S' = (S_Y, S_{\bar{Y}'})$ to satisfy the following conditions: (1) for each $s'_x \in \bar{Y}'$, $y < x \leq n$, $r'_x \leq r_x$, $p'_x = p_x$; and (2) $S_{\bar{Y}'}$ is an ECT sequence of $\bar{Y}'$ following S_Y . Therefore, if we show that $S_{\bar{Y}'}$ passes the test of optimality of Theorem 5, then $C(S') = C(S_Y, S_{\bar{Y}'}) \leq C(S_Y, S_{\bar{Y}}^*) = C(S^* | S_Y)$, where $S_{\bar{Y}}^*$ is the optimal sequence of $\bar{Y}$ following S_Y since the ready time constraints of $\bar{Y}'$ are either equal to or more relaxed than those of $\bar{Y}$. If we define $r'_1 = r_1$ and r'_x , $y \leq x \leq n$, iteratively by

$$(3) \quad r'_x = \min \{r_x, \max(C'_{x-1}, m_x)\}, \quad y < x \leq n,$$

where $m_x = \min_{y < x \leq n} r_y$, then $m_x \leq r'_x \leq r_x$, satisfying condition (1) for $\bar{Y}'$.

Furthermore, the earliest start time of s'_x ,

$$(4) \quad R'_x = \max(C'_{x-1}, r'_x) = \max[C'_{x-1}, \min\{r_x, \max(C'_{x-1}, m_x)\}].$$

Since $m_x \leq r_x$, this expression reduces to

$$(5) \quad R'_x = \max(C'_{x-1}, m_x)$$

and

$$(6) \quad C'_x = R'_x + p'_x = R'_x + p_x.$$

Therefore, (5) and (6) define R'_x and C'_x , as given by step B4 on procedure BOUND. We now show that the resulting sequence $S_{\bar{Y}'}$ follows the ECT rule. For this to be true,

$$(7) \quad C'_x = R'_x + p_x = \max(C'_{x-1}, r'_x) + p_x \leq \max(C'_{x-1}, r'_w) + p_w, \quad x < w \leq n.$$

Consider any w , $x < w \leq n$. Since $S_{\bar{Y}'}$ has the same order of $S_{\bar{Y}}$, which is an ECT sequence of $\bar{Y}$, then

$$(8) \quad C_x = \max(C_{x-1}, r_x) + p_x \leq \max(C_{x-1}, r_w) + p_w, \quad x < w \leq n.$$

Before we proceed, we observe that $R'_x \leq R_x$, $C'_x \leq C_x$ and for $x < w$, $C'_x \leq R'_w$ and $m_x \leq m_w$. From (3) and (5),

$$(9) \quad r'_x = \min(r_x, R'_x).$$

Two cases exist:

Case 1. $r_x \geq r_w$. From (8), $p_x \leq p_w$. Since $m_x \leq m_w$ and $C'_{x-1} < C'_{w-1}$ then $R'_x = \max(m_x, C'_{x-1}) \leq \max(m_w, C'_{w-1})$ and $C'_x = R'_x + p_x \leq \max(C'_{x-1}, r'_w) + p_w$.

Case 2. $r_x < r_w$. From (9) and $R'_x < C'_x \leq R'_w$, $r'_x = \min(r_x, R'_x) \leq \min(r_w, R'_w) = r'_w$. If $r_w \geq R'_x$, then $r'_w = R'_w$, implying that $\max(C'_{x-1}, r'_w) + p_w \geq r'_w + p_w = R'_w + p_w > R'_w \geq C'_x$. If $r_w < R'_w$ then $r'_w = r_w$, while $r'_x \leq r_x$. Noting that $C_{x-1} \geq C'_{x-1}$, and applying (8) we again have $\max(C'_{x-1}, r'_x) + p_x \leq \max(C'_{x-1}, r'_w) + p_w$.

Therefore, (7) is satisfied in all cases, proving that $S_{\bar{Y}'}$ orders $\bar{Y}'$ by ECT following S_Y and satisfying condition (2) for $\bar{Y}'$.

By definition, any s'_x , $y < x \leq n$, starting a block has $r'_x > C'_{x-1}$. From (3) this condition implies that $r'_x = m_x \leq m_w \leq r_x$, $x < w \leq n$, and $S_{\bar{Y}'}$ satisfies the optimality condition of Theorem 5. Therefore, $C(S') = C(S_Y, S_{\bar{Y}'}) \leq C(S_Y, S_{\bar{Y}}^*) = C(S^* | S_Y)$. Q.E.D.

REFERENCES

- [1] P. BRATLEY, M. FLORIAN AND P. ROBILLARD, *On scheduling with earliest start and due dates with application to computing bounds for the $(n/m/G/F \max)$ problem*, Naval Research Logistics Quarterly, 20 (1973), pp. 57–67.

- [2] R. CONWAY, W. MAXWELL AND L. W. MILLER, *Theory of Scheduling*, Addison-Wesley, Reading, MA, 1967.
- [3] M. I. DESSOUKY AND C. R. MARGENTHALER, *The one-machine sequencing problem with early starts and due dates*, AIIE Trans., 4 (1972), pp. 214–222.
- [4] J. K. LENSTRA, A. H. R. RINNOOY KAN AND P. BRUCKER, *Complexity of machine scheduling problems*, Ann. Discrete Math., (1977), pp. 343–362.